



Name:	Waleed Afzal
Roll No	23P-0566
Course/Section	Software Engineering/5C
Assignment	Number 3
Date:	29/11/2025

Q1 — Software Planning Scenario

Task 1 — Test Plan Development

Test Objectives

- **Functional:** Ensure real-time seat tracking, prerequisite checks, timetable conflict detection, and faculty schedule updates work correctly.
- **Non-functional:** Verify system performance during peak hours, security of student data, and scalability for future growth.

Scope

- **In-scope:** Enrollment process, seat tracking, prerequisite validation, timetable conflict detection.
- **Out-of-scope:** Payment gateway, external LMS integration.

Testing Levels

- **Unit Testing:** Individual modules like seat tracking and prerequisite validation.
- **Integration Testing:** Interaction between modules (e.g., timetable and enrollment).
- **System Testing:** Full system behavior under normal and peak loads.
- **Acceptance Testing:** Final check by university stakeholders.

Testing Approaches

- **Black-box:** Validate inputs and outputs without looking at code.
- **White-box:** Check internal logic for critical modules.

Entry & Exit Criteria

- **Entry:** Code completed, unit tests passed, test environment ready.
- **Exit:** All planned tests executed, major defects fixed, performance meets standards.

Risk Assessment & Mitigation

- Risk: Server overload during peak → Mitigation: Load testing and scalable architecture.
- Risk: Data breach → Mitigation: Strong encryption and security audits.

Resource Allocation

- Test Manager, QA Engineers, Automation Tester, Performance Tester.

Testing Schedule & Deliverables

- Week 1–2: Unit tests
- Week 3–4: Integration tests

- Week 5: System tests
 - Week 6: Acceptance tests
- Deliverables: Test cases, test reports, defect logs.

Task 2 — Review Strategy Mapping

SDLC Phase	Review Type	Why Appropriate	Participants	Expected Outcome
Requirements	Business Review	Ensures alignment with university policies	Facilitator, Author, Recorder, Reviewer	Detect missing requirements
Planning	Management Review	Confirms project goals, resources, and risks	Facilitator, Author, Recorder	Clear project plan and risk control
Design	Technical Review	Validates architecture and scalability	Facilitator, Author	Find design flaws
Development	Walkthrough	Early detection of coding issues	Author, Recorder	Minor logic errors
Testing	Inspection	Formal check of test cases and scripts	Recorder, Reviewer	Ensure complete and correct test coverage
Deployment	Management Review	Confirms readiness and risk handling	Facilitator, Author, Recorder, Reviewer	Go/No-Go decision for release

Task 3 — Constraints, Risks & Planning Decisions

Technical Constraints

1. Must support real-time updates.
2. Limited server capacity.
3. Integration with existing university database.
4. Secure login with strong password policy.
5. Mobile compatibility.

Organizational Constraints

1. Fixed budget.
2. Limited testing team.
3. Strict academic calendar deadlines.
4. Compliance with university IT policies.
5. Limited training time for staff.

Major Risks & Mitigation

1. **Peak load failure** → Perform stress testing.
2. **Data security breach** → Apply encryption and regular audits.
3. **Integration issues** → Early API testing.
4. **Schedule delays** → Use agile sprints and buffer time.
5. **Requirement changes** → Maintain change control process.
6. **Poor usability** → Conduct user acceptance testing.
7. **Hardware failure** → Backup servers and disaster recovery plan.

Q2 — Black-Box Test Design

Scenario: The CEMS login module enforces strong credential validation and account security.

Task 1 — EC & BVA Test Design

We will use **Equivalence Partitioning (EP)** and **Boundary Value Analysis (BVA)** for username, password, and failed attempts.

Partitions

- **Username**
 - Valid: 5–15 characters, alphanumeric
 - Invalid: Less than 5 or more than 15 characters, special characters only
- **Password**
 - Valid: At least 8 characters, includes uppercase, digit, and special character
 - Invalid: Missing any of these rules
- **Failed Attempts**
 - Valid: 0–4 attempts
 - Invalid: 5 or more → Account locked

Sample Test Cases (8 cases)

TC#	Input	Expected Result
1	Username = "john123", Password = "Pass@123", Attempts = 0	Login successful
2	Username = "jo", Password = "Pass@123", Attempts = 0	Error: Invalid username

TC#	Input	Expected Result
3	Username = "john123", Password = "pass1234", Attempts = 0	Error: Missing uppercase & special char
4	Username = "john123", Password = "Pass@123", Attempts = 4	Login successful
5	Username = "john123", Password = "Pass@123", Attempts = 5	Account locked
6	Username = "john1234567890123", Password = "Pass@123", Attempts = 0	Error: Username too long
7	Username = "john123", Password = "Pass1234", Attempts = 0	Error: Missing special character
8	Username = "john123", Password = "@Pass123", Attempts = 0	Login successful

Task 2 — Decision Table Construction

Conditions

- Username Valid? (Yes/No)
- Password Valid? (Yes/No)
- Failed Attempts (0–4 or ≥5)

Actions

- Success
- Error
- Account Locked

Rule	Username	Password	Attempts	System Response
1	Yes	Yes	0-4	Success
2	Yes	Yes	≥5	Account Locked
3	Yes	No	Any	Error
4	No	Yes	Any	Error
5	No	No	Any	Error

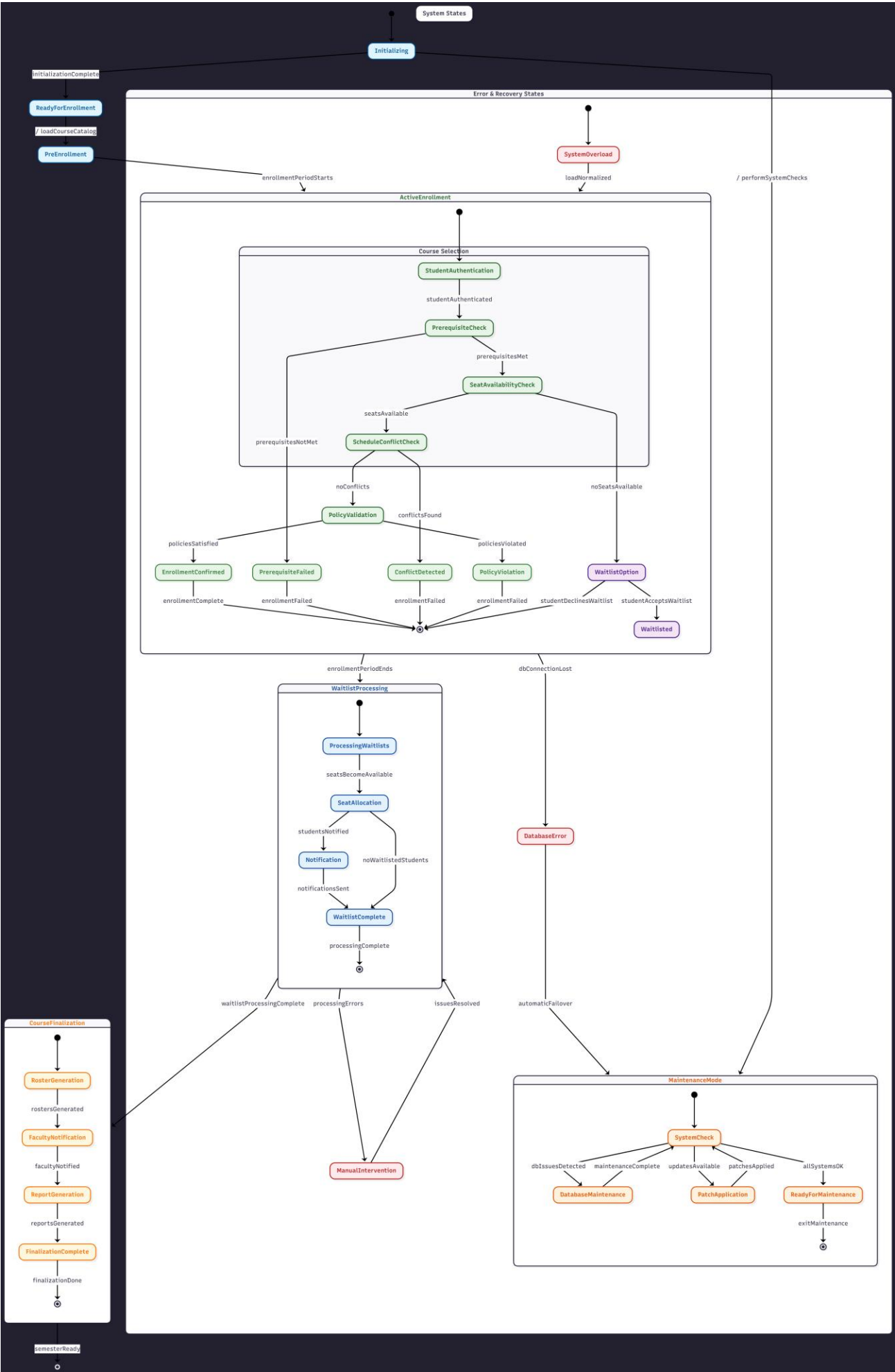
Q3 — Black-Box Test Design

Task 1 — State Diagram

Draw a state machine diagram representing:

- States
- Transitions
- Events & actions

Follow proper UML notation.



Task 2 — Multi-Step Transition Test Cases

create **3 test cases** showing how the system moves through different states based on user actions.

Test Case 1:

Step	Start State	Input	Expected Output	Finish State
1	Logged Out	Enter valid username & password	Logged In	Logged In
2	Logged In	Click Logout	Logged Out	Logged Out

Test Case 2:

Step	Start State	Input	Expected Output	Finish State
1	Logged Out	Enter wrong password 5 times	Locked	Locked
2	Locked	Admin unlocks account	Logged Out	Logged Out

Test Case 3:

Step	Start State	Input	Expected Output	Finish State
1	Logged In	No activity for 10 minutes	Inactive	Inactive
2	Active	User clicks on dashboard	Logged In	Logged In

Q4 — Test Case Writing & Defect Analysis

Task 1 — Functional & Non-Functional Test Cases

→ Functional Test Cases

1. Check Seat Availability:

- Input: Select a course
- Expected Result: Show correct available seats

2. Validate prerequisites

- Input: Enroll in advanced course
- Expected Result: Error if prerequisites are not met

3. Detect timetable clash

- Input: Add two courses with same time
- Expected Result: Show conflict message

4. Successful enrollment

- Input: Select valid course
- Expected Result: Enrollment confirmed

5. Drop a course

- Input: Remove enrolled course
- Expected Result: Course removed successfully

6. Update faculty schedule

- Input: Change class timing
- Expected Result: Updated schedule visible

→ Non-Functional Test Cases

1. Performance under load:

- **Expected Result:** System handles 1000 users

2. Performance under load:

- **Expected Result:** System handles 1000 users

3. Stress test:

- **Expected Result:** Push beyond peak load

4. Usability:

- **Expected Result:** Check UI clarity

Task 2 — Defect Classification

Defect	Type	Severity	Origin Phase	Review Type
Seat count not updating	Functional	High	Development	Walkthrough
Login fails with valid credentials	Functional	Critical	Development	Inspection
Slow response during peak	Performance	High	Testing	Management Review
Missing prerequisite check	Functional	High	Design	Technical Review
UI text overlaps	Usability	Medium	Design	Technical Review